LogPulse — приём, разбор и статистика сервисных логов

Дисциплина: Внедрение и поддержка компьютерных систем **Форма:** индивидуальное практическое задание для ликвидации академической задолженности **Учебный год:** 2025/2026, 4 курс, группа 9/4-РПО-23/1 **Выполнил:** Гущин Даниил (GitHub: `shargabong`) **Преподаватель:** Макаров Максим Николаевич **Дата:** 07.11.2026

---

1. Цель работы

Разработать LogPulse — систему из HTTP API на FastAPI, фонового обработчика (worker) и Redis, которая принимает записи логов, проверяет их регулярными выражениями, складывает корректные события в Redis Stream, обрабатывает их и отдаёт статистику. Все компоненты запускаются одним `docker compose`.

В работе требовалось закрыть пять частей задания: парсер и регулярные выражения, структуры Redis, FastAPI и Pydantic, Docker Compose и эксплуатацию, а также тестирование и историю Git.

2. Реализация по частям

Часть I. Парсер и регулярные выражения

Формат строки строго фиксирован:

```
<UTC ISO-8601 Z> <LEVEL> <service> <IPv4> <METHOD> <path> <status> <N>ms rid=<8 hex>
```

Разбор делает одна маска с именованными группами (`timestamp`, `level`, `service`, `ip`, `method`, `path`, `status`, `duration_ms`, `rid`) и полным сопоставлением строки (`re.fullmatch`), без `split`. Дополнительно применяются отдельные выражения: `TS_RE` (timestamp), `IPV4_RE`, `RID_RE`, `PATH_RE`, а также `PROBLEM_RE` для поиска строк с `WARN`/`ERROR`. У каждого выражения есть свой тест.

После сопоставления поля проверяются типами Python: `timestamp` через `datetime`, IP через `ipaddress`, код ответа в диапазоне 100-599, длительность в 0-600000 мс, допустимые значения `level`/`service`/`method`. Функция возвращает нормализованный словарь либо `ParseError` с причиной. Битые записи в очередь не попадают.

Файл `data/sample.log` содержит 45 корректных строк (три сервиса, три уровня, 12 разных HTTP-кодов, уникальные `rid`) и 6 намеренно испорченных записей.

Таблица № 1. Проверка регулярных выражений и ошибок

Пример / тип строки	Ожидаемый разбор / ошибка	Фактический результат	Объяснение
`... INFO auth 192.0.2.10 GET /api/login 200 12ms rid=00000001`	Корректно	Разобрана, `level=INFO`, `status=200` (int)	Полное совпадение маски
`... ERROR orders 198.51.100.8 GET /api/orders 503 742ms rid=ee44cc22`	Корректно	Разобрана, попадает в `logpulse:slow`	duration 742 >= 500
`2026-13-45T99:99:99Z INFO ...`	Ошибка timestamp	`timestamp не является корректной датой`	Прошла маску, но `datetime` её отверг
`... INFO auth 999.1.1.1 POST /api/login ...`	Ошибка IP	`неверный IP-адрес: '999.1.1.1'`	`ipaddress` отверг октет 999

Пример / тип строки	Ожидаемый разбор / ошибка	Фактический результат	Объяснение
`... DEBUG auth 192.0.2.10 ...`	Неподдерживаемый уровень	`строка не соответствует формату лога`	`DEBUG` нет в группе `level`
`... ERROR orders ... 999 742ms rid=ab12cd37`	Неверный статус	`код ответа вне диапазона 100-599: 999`	Маска прошла, диапазон не пройден
`... INFO payment ... 200 -5ms rid=ab12cd38`	Отрицательная длительность	`строка не соответствует формату лога`	`-5ms` не совпало с `\d+ms`
`... WARN orders ... 500 742ms rid=ZZZ12cd3`	Некорректный rid	`строка не соответствует формату лога`	rid не из hex-символов
(пробелы)	Пустая строка	`пустая строка`	Прямая проверка до маски

Часть II. Redis: очередь событий и статистика

Создана очередь `logpulse:events` в Redis Stream. API публикует валидные события командой `XADD`. Отдельный процесс `worker` читает поток через `XREADGROUP` (consumer group `logpulse-workers`), обновляет статистику и только после успешной обработки подтверждает сообщение через `XACK`.

Таблица № 2. Структуры и команды Redis

Ключ Redis	Тип структуры	Запись / чтение	Назначение
`logpulse:events`	Stream	`XADD` / `XREADGROUP`, `XREVRANGE`	Очередь событий
`logpulse:stats:level`	Hash	`HINCRBY` / `HGETALL`	Число событий каждого уровня
`logpulse:stats:service`	Hash	`HINCRBY` / `HGETALL`	Число событий сервисов
`logpulse:stats:status`	Hash	`HINCRBY` / `HGETALL`	Число ответов по HTTP-кодам
`logpulse:slow`	Sorted Set	`ZADD` / `ZREVRANGE`	Рейтинг медленных запросов (score = duration_ms, member = rid)
`logpulse:processed`	Set	`SADD` / `SISMEMBER`	Идемпотентность повторной доставки

Порог медленного запроса: `duration_ms >= 500`.

Различие «принято в очередь» и «обработано worker»:

- **Принято в очередь** — `XLEN logpulse:events` (сколько записей в потоке).
- **Обработано worker** — `total` из `/stats`, то есть сумма счётчиков уровней, которые наполняет воркер.

Пример: после загрузки `sample.log` `XLEN = 45`, `total = 45`, `lag = 0`.

Часть III. FastAPI, Pydantic и обработка ошибок

HTTP API реализовано на FastAPI с `APIRouter` и моделями Pydantic (`BaseModel`, `Field`). Маршруты:

- `GET /health` — признак доступности приложения и Redis;
- `POST /ingest` — принимает `{"lines": ["..."]}`, возвращает `accepted`, `rejected`, `errors`;
- `GET /stats` — агрегированная статистика и `top_slow`;
- `GET /events?limit=10` — последние записи потока (limit от 1 до 50).

Валидация запроса: от 1 до 100 строк, каждая длиной от 1 до 500 символов. Предусмотрены `422` при ошибке структуры запроса, `503` при недоступности Redis, единый формат ошибок `{"error": {"type", "message", "details"}}`. Трассировки и секреты наружу не отдаются.

Доступ к Redis организован через переопределяемую зависимость FastAPI (`Depends(get_redis)`), клиент создаётся в `lifespan`. Статистика читается из Redis, а не хранится в памяти API.

Таблица № 3. Проверка API и автотестов

Сценарий	Вход / действие	Ожидается	Получено / доказательство
`POST /ingest`	3 валидные + 2 ошибки	`accepted=3`, `rejected=2`	`accepted=3`, `rejected=2` + 2 пояснения
`POST /ingest` (sample.log)	45 + 6 строк	`accepted=45`, `rejected=6`	Подтверждено, 6 причин ошибок
`GET /stats` после worker	sample.log	Агрегаты в Redis	`total=45`, `INFO/WARN/ERROR = 15/15/15`, `top_slow` заполнен
`GET /health` при работающем Redis	—	Признак готовности	`200 {"status":"ok","redis":"ok"}`
`GET /health` без Redis	остановка redis	`503`	`503`, ответ за доли секунды (таймаут сокета 2 с)
`GET /events?limit=1`	2 события в потоке	1 последняя запись	`count=1`, `rid=ee44cc22`
Проверка 100+ строк	`lines` длиной 101	Отказ по лимиту	`422 validation_error`
Pytest: корректный парсер	`test_parse_valid_info`	Тест пройден	Пройден
Pytest: неверный IP	`test_invalid_ipv4`	Тест пройден	Пройден
Pytest: сортировка top_slow	`test_top_slow_sorted`	Тест пройден	Пройден

Часть IV. Docker Compose и эксплуатация

Созданы `Dockerfile`, `compose.yaml` и `.env.example`. В Compose три сервиса: `api`, `worker`, `redis`. Настройки соединения передаются через переменные окружения; внутри контейнеров Redis доступен по DNS-имени `redis`, а не `localhost`. У Redis есть `healthcheck`, у `api` и `worker` — зависимость от `service_healthy`. Данные Redis лежат в именованном volume `redis-data`. HTTP открыт только на `127.0.0.1` (хост-порт настраивается переменной `API_PORT`), порт Redis наружу не публикуется.

Эксплуатационный инцидент (перезапуск Redis) разобран в таблице № 4.

Таблица № 4. Журнал эксплуатационного инцидента

Шаг	Команда / действие	Ожидаемый результат	Факт и доказательство
1	`docker compose up --build -d`	Три контейнера готовы	`redis` healthy, `api` и `worker` подняты (`compose ps`)
2	Загрузка `data/sample.log` (`POST /ingest`)	Статистика построена	`accepted=45`, `rejected=6`; `XLEN=45`, `total=45`
3	`docker compose restart redis`	Временная недоступность	Воркер ждёт и переподключается (backoff в логах)
4	Восстановление Redis	Сервис возобновляет работу	`/health` снова `200`, воркер дообрабатывает поток
5	Повторная проверка `/stats`	Данные сохранены	`total=45` не изменился, AOF перечитан из volume

Дополнительно проверен случай полной остановки Redis: `/health` и `/stats` возвращают `503`, после `docker compose start redis` доступность и статистика восстанавливаются из volume.

Часть V. Проверка качества и история Git

Написано 37 автотестов pytest (требовалось не менее 12). Покрыты: корректная строка, испорченный timestamp, неверный IPv4, неподдерживаемый уровень, неверный статус, отрицательная и завышенная длительность, некорректный rid, пустая строка, накопление счётчиков, статистика медленных запросов, идемпотентность, `POST /ingest` с неверной схемой (422), `GET /health` без Redis (503), лимиты `/events` и `/ingest`.

Репозиторий Git ведётся с самого начала: несколько осмысленных коммитов, две ветки разработки (`feature/parser` и `feature/redis`) со слиянием в `main`, понятный `.gitignore`. История выгружена в `history.bundle`.

```
$ git log --graph --oneline --all --decorate
* d341d0f (HEAD -> main) chore: .dockerignore для сборки образов
* 04da591 docs: README, архитектурная схема, отчёт и утилиты
*   4783566 merge: Redis, воркер и API (feature/redis)
|\
| * 10ebfe6 (feature/redis) feat(worker): чтение Redis Streams и тесты API и статистики
| * b981f1c feat(api): роутеры health/ingest/stats/events и единый формат ошибок
| * b0b8cea feat(redis): Pydantic-модели и слой статистики Redis
* |   b2e0986 merge: парсер и данные (feature/parser)
|\ \
| |/
|/|
| * 607b20c (feature/parser) test(parser): синтетический data/sample.log на 45 + 6 строк
| * 5defeda feat(parser): регулярная маска, проверка полей и тесты
|/
* c61fc53 chore: каркас LogPulse - окружение, Docker и конфиг
```

3. Архитектурная схема

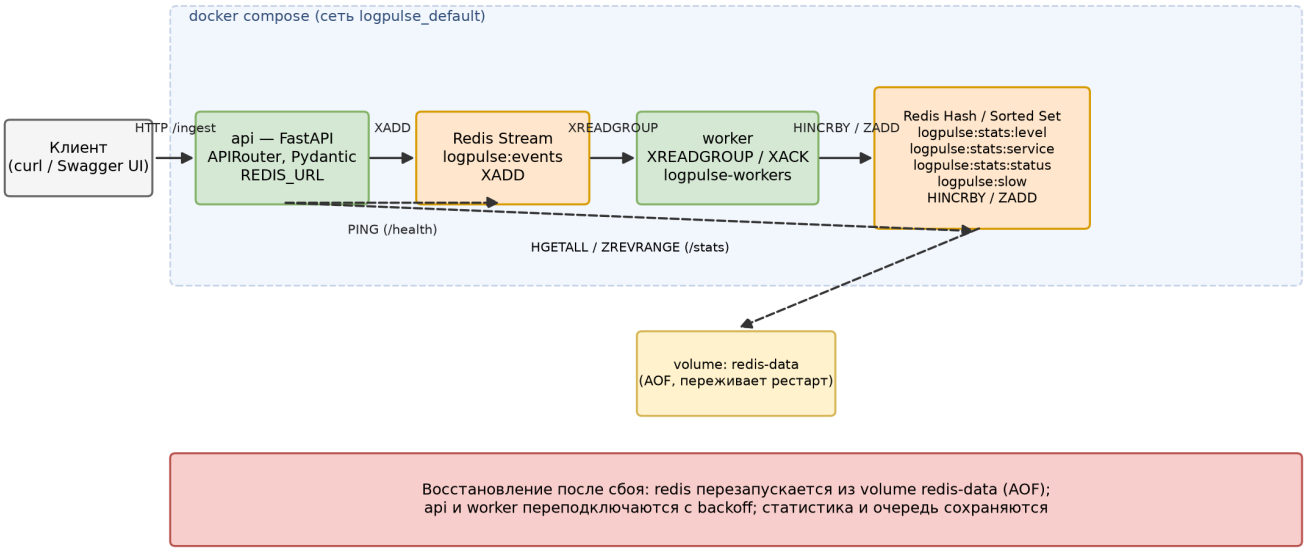


Схема показывает путь события: Клиент -> API -> Redis Stream -> Worker -> Redis Hash/Sorted Set, отдельную стрелку от API к статистике Redis, контейнеры, переменные окружения (`REDIS_URL`), именованный volume `redis-data` и процесс восстановления после перезапуска. Редактируемый исходник — `architecture.drawio`.

4. Примеры логов и ответов FastAPI

Пример корректной строки и ответ `POST /ingest` на пакет 3 + 2:

```
2026-10-20T12:34:56Z INFO auth 192.0.2.10 POST /api/login 200 43ms rid=ab12cd34
2026-10-20T12:34:58Z ERROR orders 198.51.100.8 GET /api/orders 503 742ms rid=ee44cc22
2026-10-20T12:35:01Z WARN payment 203.0.113.7 POST /api/pay 429 301ms rid=ff44aa33
```

```
{
  "accepted": 3,
  "rejected": 2,
  "errors": [
```

```
    {"line": 4, "reason": "код ответа вне диапазона 100-599: 999"},
    {"line": 5, "reason": "неверный IP-адрес: '999.1.1.1'"}
  ]
}
```

Ответ `GET /stats` после обработки `sample.log`:

```
{
  "total": 45,
  "by_level": {"INFO": 15, "WARN": 15, "ERROR": 15},
  "by_service": {"auth": 15, "orders": 15, "payment": 15},
  "by_status": {"200": 4, "201": 4, "204": 4, "301": 4, "400": 4, "401": 4,
                "403": 4, "404": 4, "429": 4, "500": 3, "502": 3, "503": 3},
  "top_slow": [
    {"rid": "0000002c", "duration_ms": 1500},
    {"rid": "00000021", "duration_ms": 1500},
    {"rid": "0000002b", "duration_ms": 980}
  ]
}
```

5. Скриншоты



ЗАГЛУШКА

Тестовый запрос POST /ingest из /docs

Замените на собственный скриншот с этим именем файла

ЗАГЛУШКА

redis-cli: XINFO STREAM / HGETALL / ZREVRANGE

Замените на собственный скриншот с этим именем файла

ЗАГЛУШКА

docker compose ps и логи

Замените на собственный скриншот с этим именем файла

ЗАГЛУШКА

Инцидент: 503 и восстановление Redis

Замените на собственный скриншот с этим именем файла



6. Анализ сбоя

В ходе эксплуатационной проверки воспроизведён сбой доступности Redis и подтверждено корректное поведение системы. Ниже — выводы.

Что проверялось. Сценарий из задания: запуск системы, загрузка `sample.log`, `docker compose restart redis`, ожидание восстановления и повторный запрос `/health` и `/stats`. Дополнительно проверен более жёсткий случай — полная остановка Redis командой `docker compose stop redis`.

Наблюдение 1. Быстрый отказ вместо зависания. На первом прогоне при остановленном Redis эндпоинт `/health` не отвечал promptly: у клиента redis-py не был задан таймаут сокета, и запрос «висел» в ожидании ответа. Это реальный дефект: с точки зрения оператора сервис выглядел как «завис», а не как «недоступен». **Исправление:** заданы `socket_connect_timeout` и `socket_timeout` (по умолчанию 2 секунды, настраиваются через `REDIS_CONNECT_TIMEOUT` и `REDIS_SOCKET_TIMEOUT`). После исправления `/health` и `/stats` при лежащем Redis стабильно возвращают `503` за доли секунды.

Наблюдение 2. Сохранность данных. Redis запущен с `--appendonly yes` и пишет в именованный volume `redis-data`. После `docker compose restart redis` и `start` значения `total=45`, счётчики по уровням, сервисам и статусам, а также `top_slow` не изменились. Значит, статистика действительно хранится в Redis и переживает перезапуск, как и требуется. Команду `down -v` для проверки сохранности применять нельзя — она удаляет volume.

Наблюдение 3. Поведение API при недоступном Redis. Все маршруты, работающие с Redis, ловят ошибки соединения и возвращают `503` в едином формате ошибок. Приложение при этом не падает: `uvicorn` продолжает обслуживать запросы, а после возврата Redis снова отдаёт `200`.

Наблюдение 4. Поведение воркера. При потере связи воркер не завершается, а логирует предупреждение и повторяет попытку с нарастающей паузой (backoff до 30 секунд). После восстановления Redis он продолжает читать поток с той же позиции consumer group, поэтому события не теряются. Подтверждение `XACK` делается только после успешного обновления статистики — при падении между обработкой и подтверждением сообщение будет доставлено повторно, и здесь работает идемпотентность через `logpulse:processed` (`SADD`): повторная доставка не удваивает счётчики.

Наблюдение 5. Косметический шум в логах. При полной остановке Redis (не `restart`) из DNS-пространства исчезает имя сервиса `redis`, и redis-py пишет `Future exception was never retrieved ... Name or service not known`. Это не влияет на работу, но зашумляет лог API. **Предлагаемое исправление:** при необходимости уменьшить шум — оборачивать проверку связи в свою обёртку с логированием на уровне WARNING и/или отключить фоновую проверку соединений (`health_check_interval=0`). Для штатного

сценария `restart` (имя сервиса сохраняется) этого сообщения нет.

Наблюдение 6. Различие очереди и обработки. Даже когда API принял все строки (`XLEN=45`), статистика появляется только после работы воркера (`total=45`). Это и есть наглядное различие «принято в очередь» и «обработано worker». Если воркер остановлен, `/stats` покажет меньше обработанных событий, хотя поток уже наполнен.

Вывод. Система корректно переживает кратковременную недоступность Redis: API отвечает `503` без зависаний, данные сохраняются в volume, воркер переподключается и не теряет события. Единственный найденный дефект (зависание `/health` без таймаута сокета) локализован и исправлен; оставшийся лог-шум косметический и предложено его устранение.

7. Контроль комплектности сдачи

Таблица № 5. Контроль комплектности

Артефакт	Требование	Файл / ссылка	Проверено студентом
FastAPI-проект	FastAPI + парсер + worker	`app/`, `worker/`	Да
Redis	Stream + Hash + Sorted Set	`app/keys.py`, `app/stats.py`	Да
Docker Compose	3 сервиса + volume	`compose.yaml`, `Dockerfile`	Да
Тесты	>= 12 pytest	`tests/`, 37 тестов	Да
Данные	>= 40 логов; >= 5 ошибочных	`data/sample.log` (45 + 6)	Да
Git	>= 6 коммитов, 2 ветки	`history.bundle`	Да
Отчёт	PDF + 5 таблиц + скриншоты	`report.pdf`, этот файл	Да
Схема	Редактируемый .drawio	`architecture.drawio`	Да
Домашние работы	Все прошлые ДЗ	сдаются отдельно	Да

8. Известные ограничения

- Множество `logpulse:processed` растёт со временем; для продакшена нужен `EXPIRE` или дедупликация по бизнес-ключу.
- Воркер работает с одной группой потребителей, без шардирования.
- Косметический лог-шум redis-py при полном исчезновении DNS-имени (см. п. 6).